In the pictures below, we will only show element's priorities or, to see it in another way, we assume that elements and priorities are the same. This is just for the sake of space and clarity in diagrams, but it also highlights an important characteristic of heaps: in all their methods, we only need to access and move priorities. This can be important when elements are large objects: especially if they are so large that won't fit in cache/memory or for any reason they are stored on disk, the concrete implementation can just store and access a reference to the elements and its priority.

Now, before delving into the API methods, we need to define two helper functions that will be used to reinstate the heap properties whenever a change is performed. The possible changes for a heap are

- Adding a new element to the heap.
- Removing an element of the heap.
- Updating an element's priority.

Any of these operations can cause a heap element to be smaller than its parent, or larger than (one of) its children.

2.6.1 BubbleUp

When an element has a larger priority than its parent, it is necessary to call the `bubbleUp` method. Figure 2.6 shows an example based on our task management tool of such a situation that we will use as a reference: the element at index 7 is larger than its parent (at index 2) and thus these two elements need to be swapped.

Listing 2.2 The bubbleUp method